Midterm Study Guide

Programming Language Paradigms

- **Imperative:** Focus on step-by-step instructions and modifying program state (e.g., C, Java, Python).
- **Functional:** Emphasize evaluating mathematical functions and immutability (e.g., Lisp, Haskell, ML).
- **Logical:** Express programs as logical rules and relationships (e.g., Prolog).
- **Object-Oriented:** Organize code into objects with data and methods (e.g., Java, C++, Python).

Key Differences

- Imperative languages rely on changing variables and assignments.
- Functional languages avoid side effects and mutable data.
- Logical languages use logical inference to find solutions.
- Object-oriented languages combine data and methods into objects.

Turing Completeness

- A language is Turing complete if it can perform any computation that a Turing machine can.
- Most general-purpose languages are Turing complete.
- The Church-Turing thesis states that any problem solvable by one Turing-complete language is solvable by any other.

BNF Grammar Notation

- BNF (Backus-Naur Form) is a notation for expressing grammars.
- It uses rules like `<non-terminal> ::= sequence of terminals and non-terminals`
- Examples:
  - `<expr> ::= <expr> + <term> | <term>`
  - `<term> ::= <term> * <factor> | <factor>`

Parse Trees

- Parse trees are hierarchical representations of the syntactic structure of a string according to a grammar.
- They show how a string is derived from the start symbol using the grammar rules.

Ambiguity in Grammars

- A grammar is ambiguous if it allows multiple parse trees for the same string.

- Ambiguity can lead to different interpretations of a program.
- Techniques to remove ambiguity:
  - Precedence: Define the order of operations for operators.
  - Associativity: Define how operators of the same precedence are grouped.
  - Rewriting grammar rules: Introduce new non-terminals or modify existing rules to enforce a specific parse tree.

Example: Removing Ambiguity

- Original grammar (ambiguous):

```
<expr> ::= <expr> + <expr> | <expr> * <expr> | <id>
```

- Modified grammar (unambiguous, with precedence and left-associativity):

```
<expr> ::= <expr> + <term> | <term>
<term> ::= <term> * <factor> | <factor>
<factor> ::= <id>
```

The Classical Compilation Sequence

1. **Creation:** Writing source code in a high-level language.
2. **Compilation:** Translating source code into assembly language.
3. **Assembly:** Translating assembly language into machine code (object file).
4. **Linking:** Combining object files and resolving external references.
5. **Loading:** Loading the executable into memory.
6. **Running:** Executing the program.

Load Time

- Load time is when the program is loaded into memory before execution.
- The loader performs tasks like:
  - Resolving final addresses of variables and functions.
  - Setting up the program's memory space.
  - Performing dynamic linking (if applicable).

Beta Reduction in Lambda Calculus

- Beta reduction is a fundamental operation in lambda calculus, which is a formal system for expressing computation based on function abstraction and application.
- It involves substituting the argument of a function for the bound variable in the function's body.
- Example:
  - `(λx. x + 1) 2` (Apply the function to the argument 2)
  - `2 + 1` (Substitute 2 for `x` in the function body)
  - `3` (Evaluate the expression)

SML (Standard ML)

- A functional programming language with strong static typing.
- Key features:
  - Functions as first-class citizens
  - Immutability
  - Recursion
  - Pattern matching
  - Type inference